

FULL STACK DEVELOPMENT

FASE 4 | AULA 03 -

REACT NAVIGATION

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	22
REFERÊNCIAS.....	23

EMSE

O QUE VEM POR AÍ?

Nesta aula, você terá um contexto de como funciona o desenvolvimento de rotas para aplicativos mobile. Abordaremos boas práticas, vantagens e desvantagens de alguns dos métodos de navegação, passagem de parâmetros por rota configuração e aplicaremos um dos métodos na prática.



HANDS ON

Abordaremos o uso do StackNavigator juntamente com o expo, a configuração que devemos fazer para utilizarmos as rotas, como fazer a passagem de parâmetros através das nossas rotas, criaremos e estilizaremos nossa nova tela.



SAIBA MAIS

Segue abaixo o código utilizado no Hands on:

```
npm install @react-navigation/native-stack
```

```
import { StatusBar } from 'expo-status-bar';
import React, { useState } from 'react';
import { Alert, Button, StyleSheet, Text, TextInput, View, TouchableOpacity } from 'react-native';
import Header from '../shared/header';
import { LinearGradient } from 'expo-linear-gradient';
import { useNavigation } from '@react-navigation/native';

export default function Login() {
  const [user, setUser] = useState("");
  const [password, setPassword] = useState("");
  const navigation = useNavigation();

  const handleButtonPress = () => {
    Alert.alert(
      'Credenciais Digitadas',
      `Usuário: ${user}\nSenha: ${password}`,
      [
        {
          text: 'OK',
          onPress: () => console.log('OK Pressed'),
          style: 'cancel',
        },
      ],
    );
  };
}
```

```
      accessibilityLabel: 'Fechar Alerta',
    },
  ],
  { cancelable: true }
);
navigation.navigate('screens/Home/index',{userName: user})
};
return (
<LinearGradient
  colors={['#87CEEB', '#FFFFFF']}
  style={styles.container}
  start={{ x: 0.5, y: 0 }}
  end={{ x: 0.5, y: 1 }}
>
  <Header/>
  <View style={styles.content}>
    <Text>Digite seu usuário</Text>
    <TextInput style={styles.input}
      onChangeText={({inputText})=>{setUser(inputText)}}
      value={user}
      accessibilityLabel="Campo de usuário"
    />
    <TextInput/>
    <Text>Digite sua senha</Text>
    <TextInput style={styles.input}
      onChangeText={({inputText})=>{setPassword(inputText)}}
    />
  </View>
</LinearGradient>
);
}
```

```
        value={password}
        accessibilityLabel="Campo de senha"
      />

      <StatusBar style="auto" />

      <TouchableOpacity      style={styles.button}    onPress={handleButtonPress}
activeOpacity={0.1}>

        <Text>Enviar</Text>
      </TouchableOpacity>
    </View>
  </LinearGradient>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
  },
  content: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  input: {
    height: 40,
```

```
borderColor: 'gray',
borderWidth: 1,
marginBottom: 10,
paddingHorizontal: 10,
width: '100%',
borderRadius: 25,
},
button: {
  backgroundColor: 'inherit',
  borderColor: 'grey',
  borderWidth: 1,
  width: '50%',
  alignItems: 'center',
  padding: 5,
}
});

import { createNativeStackNavigator } from '@react-navigation/native-stack'

import Login from '../screens/Login'
import { Home } from '../screens/Home'

const { Navigator, Screen } = createNativeStackNavigator()

export function AppRoutes() {
  return(
```



```
<Navigator screenOptions={{ headerShown: false }}>
  <Screen name="screens/Login/index"
    component={Login}/>
  <Screen name="screens/Home/index"
    component={Home}/>
</Navigator>
)
}
```

```
import { NavigationContainer } from '@react-navigation/native'

import { AppRoutes } from './app.routes'

export function Routes(){
  return(
    <NavigationContainer independent={true}>
      <AppRoutes/>
    </NavigationContainer>
  )
}
```

```
import { Feather } from "@expo/vector-icons";
import { LinearGradient } from "expo-linear-gradient";
import { View, Text, StyleSheet } from "react-native";

export function WeatherCardInfo({user, temperature}){
  const actualDate = new Date()
```

```
const dateFormatted = actualDate.toLocaleDateString('pt-BR', {
  day: '2-digit',
  month: '2-digit',
  year: 'numeric',
});

return(
  <LinearGradient style={styles.container}
    colors={['#87CEEB', '#ff23']}
    start={{ x: 0.5, y: 0 }}
    end={{ x: 0.5, y: 1 }}
  >
    <View style={styles.header}>
      <Text style={styles.paragraph}>Boas Vindas {user}</Text>
      <Text style={styles.paragraph}>{dateFormatted}</Text>
    </View>
    <View style={styles.weather}>
      <Feather name="sun" size={50} color="#fff" />
      <Text style={styles.weatherText}>{temperature}</Text>
    </View>
  </LinearGradient>
)
}

const styles = StyleSheet.create({
  container: {
    alignItems: 'center',
    justifyContent: 'center',
```

```
padding: 20,
width: '100%',
},
header:{
  display:'flex',
  flexDirection: 'row',
  justifyContent:'space-between',
  width: '100%'
},
weather:{
  marginTop:20,
  display:'flex',
  justifyContent:'center',
  alignItems:'center'
},
weatherText:{
  fontSize: 20,
  fontWeight: 'bold',
  color:'#fff'
},
paragraph: {
  fontSize: 18,
  textAlign: 'center',
  color:'#fff'
},
});
```

```
import { Feather } from "@expo/vector-icons";
import { LinearGradient } from "expo-linear-gradient";
import { View, Text, StyleSheet } from "react-native";

export function TemperatureCard({temperature}){
  return(
    <LinearGradient style={styles.container}
      colors={['#87CEEB', '#ff23']}
      start={{ x: 0.5, y: 0 }}
      end={{ x: 0.5, y: 1 }}
    >
      <View style={styles.weather}>
        <Feather name="sun" size={50} color="#fff" />
        <Text style={styles.weatherText}>{temperature}</Text>
      </View>
    </LinearGradient>
  )
}

const styles = StyleSheet.create({
  container: {
    alignItems: 'center',
    justifyContent: 'center',
    padding: 20,
    width: '40%',
  },
});
```

```
header:{
  display:'flex',
  flexDirection: 'row',
  justifyContent:'space-between',
  width: '100%'
},
weather:{
  marginTop:20,
  display:'flex',
  justifyContent:'center',
  alignItems:'center'
},
weatherText:{
  fontSize: 20,
  fontWeight: 'bold',
  color:'#fff'
},
paragraph: {
  fontSize: 18,
  textAlign: 'center',
  color:'#fff'
},
});
```

```
import { Routes } from "@routes";
export default function RootLayout () {
  return (
```

```
<Routes/>  
  
);  
}
```

React Navigation

O React Navigation é uma biblioteca popular e altamente flexível para lidar com a navegação em aplicativos React Native. Ela foi desenvolvida para simplificar a implementação de fluxos de navegação complexos, proporcionando uma experiência de usuário fluida e intuitiva em dispositivos móveis.

Ele é mantido pela comunidade React Native e oferece suporte para vários tipos de navegação, personalização extensiva e integração com outras ferramentas de gerenciamento de estado, como Redux.

Sobre o React Navigation, temos alguns pontos importantes:

Tipos de navegação

StackNavigator

O StackNavigator é um dos tipos de navegação mais comuns e versáteis disponíveis no React Navigation para criar fluxos de navegação em aplicativos React Native. Vamos explorar mais detalhadamente o StackNavigator e suas características:

Pilha de telas

O StackNavigator gerencia a navegação através de uma pilha de telas. Isso significa que cada vez que uma nova tela é exibida, ela é empilhada sobre a tela anterior na hierarquia, formando uma pilha de navegadores.

Esse modelo é semelhante à navegação em um navegador da web, onde você pode avançar (empilhar) para uma nova página e voltar (desempilhar) para a página anterior.

Funcionamento básico

Para utilizar um StackNavigator, você precisa primeiro configurá-lo em um componente principal do seu aplicativo, geralmente no arquivo App.js ou NavigationContainer.js.

Cada tela do seu aplicativo é representada por um componente React que é associado a uma chave única (como um nome de rota) dentro do StackNavigator.

Navegação entre telas

Para navegar entre as telas empilhadas, você pode usar o objeto navigation fornecido pelo React Navigation. Por exemplo, navigation.navigate('NomeDaTela') é usado para navegar para uma nova tela na pilha.

Você também pode utilizar outras ações de navegação, como goBack() para voltar para a tela anterior na pilha ou push() para empilhar uma nova tela.

Passagem de parâmetros

Uma característica importante do StackNavigator é a capacidade de passar parâmetros entre as telas durante a navegação. Isso permite enviar dados específicos ou configurações para a próxima tela na pilha.

Por exemplo, você pode passar parâmetros ao chamar navigation.navigate('Infoltem', { itemId: 1 }) e acessá-los na tela de destino através de route.params.itemId.

Header (cabeçalho) personalizável

O StackNavigator inclui uma header (cabeçalho) por padrão em cada tela, exibindo o título da tela e botões de navegação. Esse cabeçalho pode ser personalizado de várias maneiras: você pode definir um título personalizado para cada tela, adicionar botões personalizados ao cabeçalho e modificar o estilo conforme necessário.

Gestão da navegação

O estado da navegação, incluindo a pilha de telas e os parâmetros associados, é gerenciado pelo React Navigation de forma transparente. Isso facilita a sincronização e o controle da navegação em diferentes partes do seu aplicativo.

Além disso, o React Navigation fornece eventos de ciclo de vida da navegação que podem ser utilizados para executar ações específicas ao entrar ou sair de uma tela.

DrawerNavigator

O DrawerNavigator é um tipo de navegador (navigator) oferecido pela biblioteca React Navigation, que é amplamente utilizada para gerenciar a navegação em aplicativos React Native.

A principal característica do DrawerNavigator é a capacidade de criar um menu lateral que pode ser aberto deslizando da borda esquerda ou direita da tela (dependendo da configuração e do layout do aplicativo), permitindo aos usuários acessar diferentes seções ou funcionalidades do aplicativo de maneira intuitiva. Esse menu normalmente contém links ou itens de navegação para telas específicas do aplicativo, como páginas de configurações, perfil do usuário, mensagens, entre outras.

Funcionamento do Drawer Navigator

Esse menu geralmente contém links para diferentes telas ou seções do aplicativo, proporcionando uma forma intuitiva de navegação para o usuário.

Configuração básica

Para utilizar o DrawerNavigator, você precisa importá-lo do React Navigation em seu componente principal, geralmente no arquivo App.js ou em um arquivo dedicado de navegação. Em seguida, você define as diferentes telas do seu aplicativo como componentes React e configura o DrawerNavigator para incluir essas telas no menu lateral.

Abertura e fechamento do menu lateral

O menu lateral do DrawerNavigator, por padrão, pode ser aberto deslizando da borda esquerda ou direita da tela, ou através de um botão ou gesto específico, dependendo da configuração e das preferências de design.

Personalização do menu lateral

O DrawerNavigator oferece várias opções de personalização para o menu lateral. Você pode definir o conteúdo do menu, como itens de navegação, cabeçalho, rodapé e até mesmo componentes personalizados. Além disso, é possível personalizar o estilo do menu, como cores, ícones, fontes e espaçamento, para se adequar ao design do seu aplicativo.

Navegação no menu lateral

Ao configurar telas no DrawerNavigator, cada uma delas se torna um item de navegação no menu lateral. Quando o usuário seleciona um item, a tela correspondente é exibida no conteúdo principal.

A navegação entre as telas é tratada automaticamente pelo React Navigation, permitindo que você se concentre na estrutura e funcionalidade do menu lateral.

Gestão do estado de navegação

Assim como outros tipos de navegadores do React Navigation, o estado de navegação do DrawerNavigator é gerenciado internamente pela biblioteca. Isso inclui o controle da abertura e fechamento do menu lateral.

Você pode acessar e modificar o estado de navegação através do objeto navigation fornecido nos componentes das telas do menu lateral.

Personalização avançada

Além das opções básicas de personalização, o DrawerNavigator suporta funcionalidades avançadas, como a capacidade de adicionar gatilhos de abertura do menu lateral personalizados, animações personalizadas durante a transição e configurações específicas para diferentes plataformas (iOS e Android).

TabNavigator

O TabNavigator é um dos tipos de navegadores (navigators) disponíveis na biblioteca React Navigation. Ele é projetado para criar uma barra de navegação baseada em abas na parte inferior (ou superior) da tela, permitindo que os usuários alternem entre diferentes seções ou funcionalidades do aplicativo de forma intuitiva.

Ele oferece uma maneira conveniente de organizar e apresentar conteúdos distintos em abas separadas, cada uma representando um aspecto específico do

aplicativo. Por exemplo, em um aplicativo de redes sociais, você pode ter abas para o feed de notícias, mensagens, perfil do usuário e configurações.

Funcionamento do Tab Navigator

O TabNavigator permite que você organize diferentes telas do seu aplicativo em abas na parte inferior (ou superior) da tela. Cada aba representa uma seção ou funcionalidade específica do aplicativo. Os usuários podem alternar entre as abas tocando nelas, proporcionando uma navegação intuitiva e rápida.

Configuração básica

Para utilizar o TabNavigator, você precisa importá-lo do React Navigation em seu componente principal, geralmente no arquivo App.js ou em um arquivo dedicado de navegação. Em seguida, você define as diferentes telas do seu aplicativo como componentes React e configura o TabNavigator para incluí-las como abas.

Personalização das abas

O TabNavigator oferece várias opções de personalização para as abas. Você pode definir o ícone e o rótulo de cada aba, assim como o estilo geral das abas, como cores, fontes e espaçamento.

Além disso, você pode adicionar mais abas conforme necessário e reordená-las para refletir a estrutura desejada do seu aplicativo.

Navegação entre abas

A navegação entre as abas no TabNavigator é simples e intuitiva. Os usuários podem alternar entre as abas tocando nelas na barra de navegação.

O React Navigation gerencia automaticamente o estado de navegação das abas, mantendo a posição correta quando o usuário volta para a tela anterior ou navega entre abas.

Conteúdo dinâmico nas abas

Cada aba do TabNavigator pode ter seu próprio conteúdo dinâmico e interativo. Isso permite que você exiba informações diferentes ou funcionalidades específicas em cada aba.

Por exemplo, você pode ter uma aba "Notificações" que exibe uma lista de notificações em tempo real e outra aba "Perfil" que mostra informações do perfil do usuário.

Navegação aninhada

O TabNavigator pode ser combinado com outros tipos de navegadores, como o StackNavigator, para criar navegação aninhada dentro das abas. Isso é útil quando você precisa de fluxos de navegação mais complexos dentro de cada seção do aplicativo.

Por exemplo, em uma aba "Configurações", você pode ter um StackNavigator para navegar entre diferentes telas de configuração.

Personalização avançada

Além das opções básicas de personalização, o TabNavigator suporta funcionalidades avançadas, como a capacidade de adicionar mais de uma barra de navegação (por exemplo, uma na parte inferior e outra na parte superior), personalização de transições entre abas e animações específicas para cada aba.

Configuração e uso

Para utilizar o React Navigation, você precisa primeiro instalar a biblioteca em seu projeto React Native. Isso é feito através do gerenciador de pacotes npm ou yarn.

Depois de instalado, você pode configurar os diferentes tipos de navegadores (Stack Navigator, Drawer Navigator, Tab Navigator) em seu componente principal, geralmente no arquivo App.js ou NavigationContainer.js.

Cada tela do seu aplicativo é representada por um componente React, e a navegação entre essas telas é feita utilizando os métodos e componentes fornecidos pelo React Navigation, como `navigation.navigate('NomeDaTela')` para navegar para uma tela específica.

Integração com Redux

- **Middleware:** o React Navigation fornece um middleware específico para o Redux chamado `createreactnavigationreduxmiddleware`. Esse middleware

é utilizado para conectar a navegação do React Navigation ao Redux, permitindo que o estado de navegação seja gerenciado pelo Redux de forma eficiente.

- Reducer de navegação: no Redux é necessário configurar um reducer para gerenciar o estado de navegação. O React Navigation fornece uma função `createNavigationReducer` para isso, que permite combinar o estado de navegação com outros reducers do seu aplicativo.
- Conexão do store ao navegador: após configurar o middleware e o reducer de navegação, é preciso conectar o store Redux ao navegador do React Navigation utilizando o componente `createReduxContainer`. Isso permite que o estado de navegação seja acessível em componentes conectados ao redux.

Integração com Context API

- Criação do contexto: utilizando a Context API do React, é possível criar um contexto para o estado de navegação. Isso permite compartilhar o estado de navegação entre diferentes componentes sem a necessidade de passar props manualmente.
- Provedor de navegação: é necessário criar um provedor de navegação que utilize o contexto criado para fornecer o estado de navegação aos componentes filhos que precisam acessá-lo. Esse provedor pode ser colocado em um componente de alto nível do seu aplicativo.
- Utilização do contexto: nos componentes que necessitam acessar o estado de navegação, é possível utilizar o hook `useContext` para consumir o contexto criado. Isso permite acessar o estado de navegação de forma direta sem a necessidade de passar props através da árvore de componentes.

Ambas as abordagens oferecem formas eficazes de integrar o estado de navegação do React Navigation com o Redux ou a Context API, proporcionando uma maneira centralizada e gerenciável de lidar com a navegação em aplicativos React Native. A escolha entre Redux e Context API dependerá das necessidades

específicas do seu aplicativo e da estrutura de gerenciamento de estado que você deseja adotar.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos sobre o desenvolvimento de rotas para aplicativos móveis, incluindo boas práticas e uma análise das vantagens e desvantagens de diferentes métodos de navegação.

Também tivemos conteúdo sobre a passagem de parâmetros por rota, uma técnica fundamental para transmitir dados entre diferentes telas em um aplicativo. Além disso, nos envolvemos na configuração e aplicação prática de um dos métodos de navegação estudados, proporcionando uma experiência hands-on para consolidar o conhecimento teórico adquirido.

Durante esta aula, foi possível compreender não apenas como criar rotas eficientes e intuitivas em aplicativos móveis, mas também como escolher o método de navegação mais adequado para cada situação, considerando as necessidades específicas do projeto. Isso incluiu a avaliação de aspectos como desempenho, usabilidade e flexibilidade de cada método, preparando cada estudante para tomarem decisões informadas ao desenvolverem aplicativos.

REFERÊNCIAS

EXPO. Documentação oficial. Disponível em: <https://docs.expo.dev>. Acesso em: 19 jul. 2024.

REACT NATIVE. Documentação Oficial. Disponível em: <https://reactnative.dev/>. Acesso em: 19 jul. 2024.

REACT NAVIGATION. Documentação oficial. Disponível em: <https://reactnavigation.org>. Acesso em: 19 jul. 2024.

PALAVRAS-CHAVE

Palavras-chave: React Native. React Navigation. StackNavigation.

EMSE

POS TECH